

TD4 - Assembleur

Que fait ce code ?

Pour chacune des fonctions écrites en assembleur suivantes :

- Déroulez l'exécution, en regardant l'état de la pile, et des registres,
- Déduisez-en l'algorithme réalisé par la fonction.

Fonction 1 :

```
1  .LC0:
2      .string "%d\n"
3  mystery1:
4      push    rbp
5      mov     rbp, rsp
6      sub     rsp, 16
7      mov     DWORD PTR [rbp-4], 1
8      jmp     .L2
9  .L3:
10     mov     eax, DWORD PTR [rbp-4]
11     mov     esi, eax
12     lea     rdi, [rip + .LC0]
13     mov     eax, 0
14     call    printf
15     add     DWORD PTR [rbp-4], 1
16  .L2:
17     cmp     DWORD PTR [rbp-4], 99
18     jle     .L3
19     add     rsp, 16
20     pop     rbp
21     ret
```

Fonction 2 :

```
1  mystery2:
2      push    rbp
3      mov     rbp, rsp
4      sub     rsp, 16
5      mov     DWORD PTR [rbp-4], edi
6      cmp     DWORD PTR [rbp-4], 0
7      jne     .L2
8      mov     eax, 0
9      jmp     .L3
10 .L2:
11      mov     eax, DWORD PTR [rbp-4]
12      sub     eax, 1
13      mov     edi, eax
14      call    mystery2
15      mov     edx, DWORD PTR [rbp-4]
16      add     eax, edx
17 .L3:
18      add     rsp, 16
19      pop     rbp
20      ret
```

Rappels et compléments

Dans ce TD vous écrirez des fonctions en assembleur, qui reçoivent des arguments et retournent une valeur.

Nous utiliserons la convention d'appel Linux x64, ce qui signifie que les arguments seront passés dans les registres (dans l'ordre) `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`. La valeur de retour sera placée dans `rax`.

Lors de l'appel à une fonction, l'adresse du haut de la pile (qui se trouve dans le registre `rsp`) doit être aligné sur une adresse multiple de 16 octets. Si ce n'est pas le cas, vous devez « padder » la pile pour atteindre une telle adresse avant d'appeler une fonction.

Exercice 1 : Premières fonctions

Réalisez les fonctions `add` et `multiply` permettant d'additionner et multiplier deux nombres entiers passés en paramètre.

Exercice 2 : Factorielle

Ecrire une fonction récursive `factorielle` permettant de calculer la factorielle d'un nombre entier. Comment évolue la call stack pendant l'exécution ? Que contient-elle ? Quel(s) changement(s) dans votre fonction pourrait impacter l'évolution de la call stack ?

Exercice 3 : Boucle for

Ecrire une fonction permettant de calculer la somme des n premiers entiers non nuls en utilisant une boucle.

Exercice 4 : Operateur modulo

Ecrivez une fonction permettant de calculer le reste d'un entier modulo un autre, **sans utiliser d'instruction de division**. On suppose que les arguments de la fonction sont positifs.